

Ingeniería en Computación

ELECTRÓNICA DIGITAL

Teórico 2

Funciones Lógicas y Álgebra de Boole



Universidad Nacional
de **Río Negro**

Compuertas lógicas

- El término *compuerta* se usa para describir un circuito que realiza una operación lógica básica. Es un circuito de transistores.
- Vamos a ver NOT, AND, OR, NAND, NOR, XOR y XNOR
- Para cada una vamos a ver:
 - Símbolo lógico ANSI/IEEE 91-1991
 - Expresión Funcional
 - Funcionamiento – diagrama de tiempos.
 - Tabla de verdad

Compuerta NOT (Inversor/Complemento/Negador)

Símbolo y Expresión Funcional

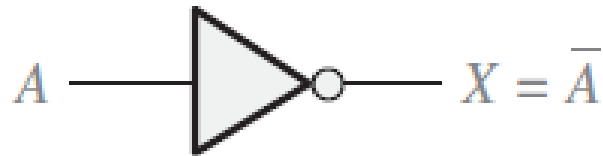


Tabla de Verdad

Entrada	Salida
BAJO (0)	ALTO (1)
ALTO (1)	BAJO (0)

Función

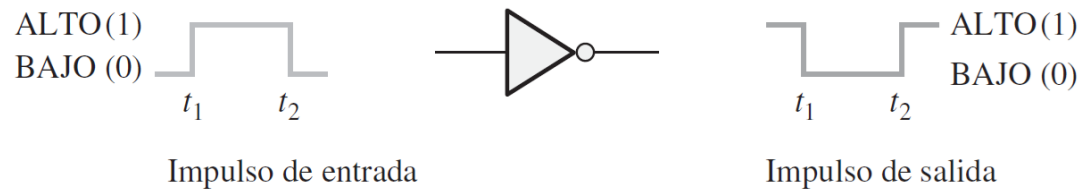
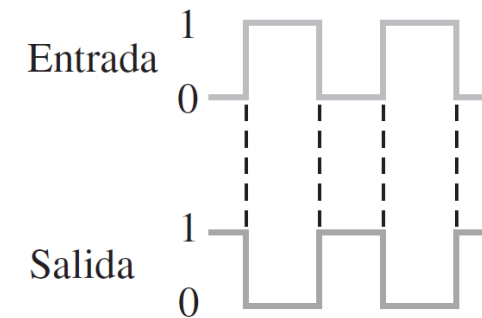
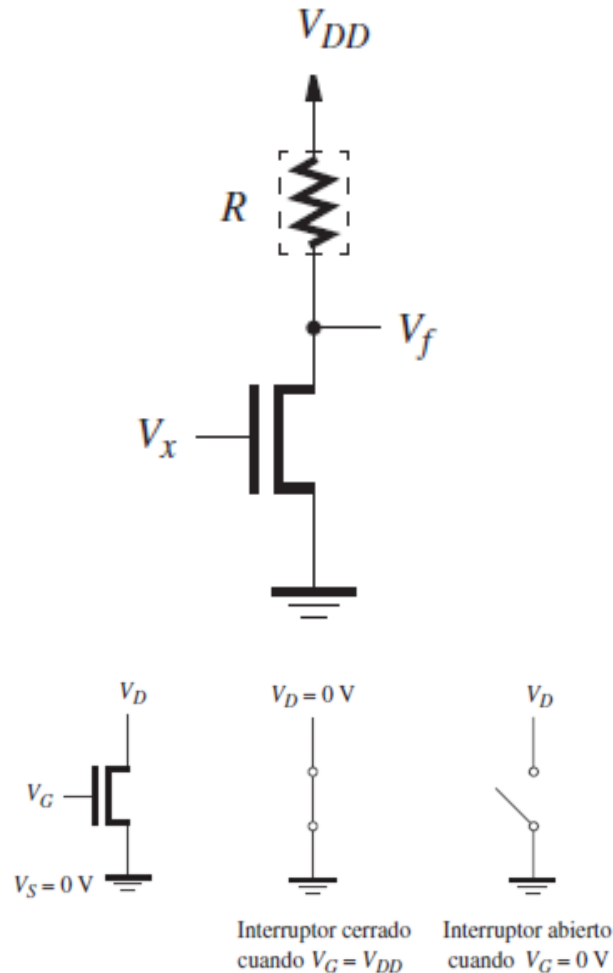


Diagrama de tiempos



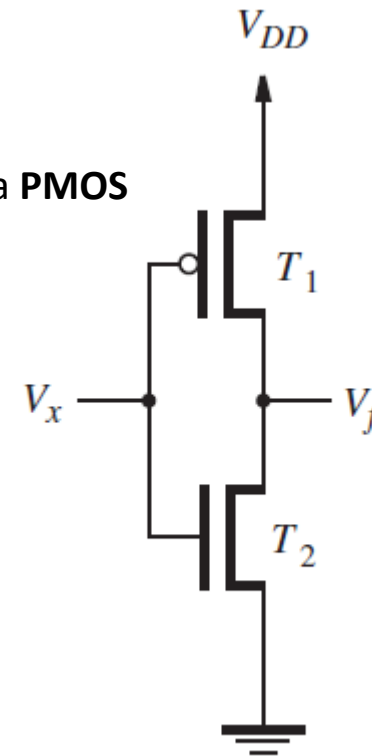
Implementación Básica de Puertas: NOT

■ Tecnología NMOS



■ Tecnología CMOS

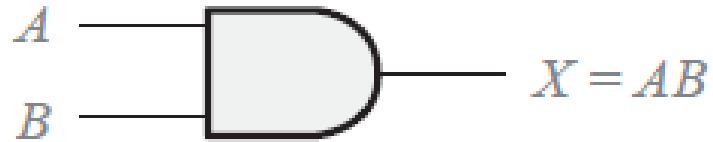
■ Tecnología PMOS



x	T_1	T_2	f
0	on	off	1
1	off	on	0

Compuerta AND (Producto Lógico)

Símbolo y Expresión Funcional



La **compuerta AND** genera una salida a nivel ALTO *sólo* cuando *todas* las entradas están a nivel ALTO.

Tabla de Verdad

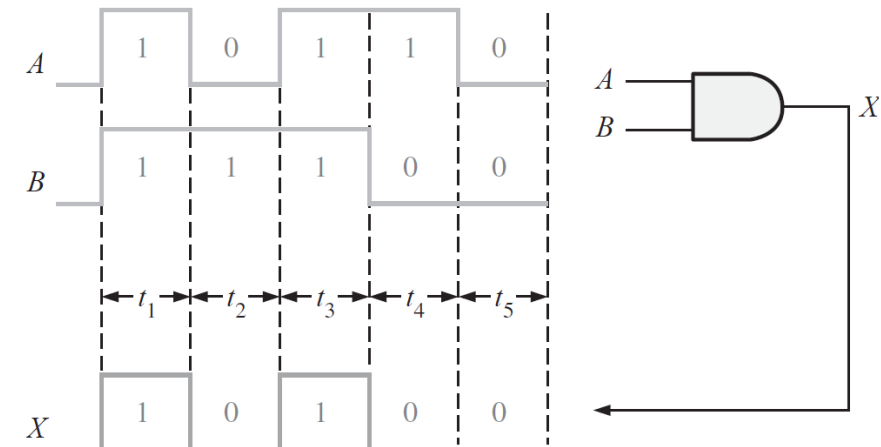
Entradas		Salida
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

1 = ALTO, 0 = BAJO

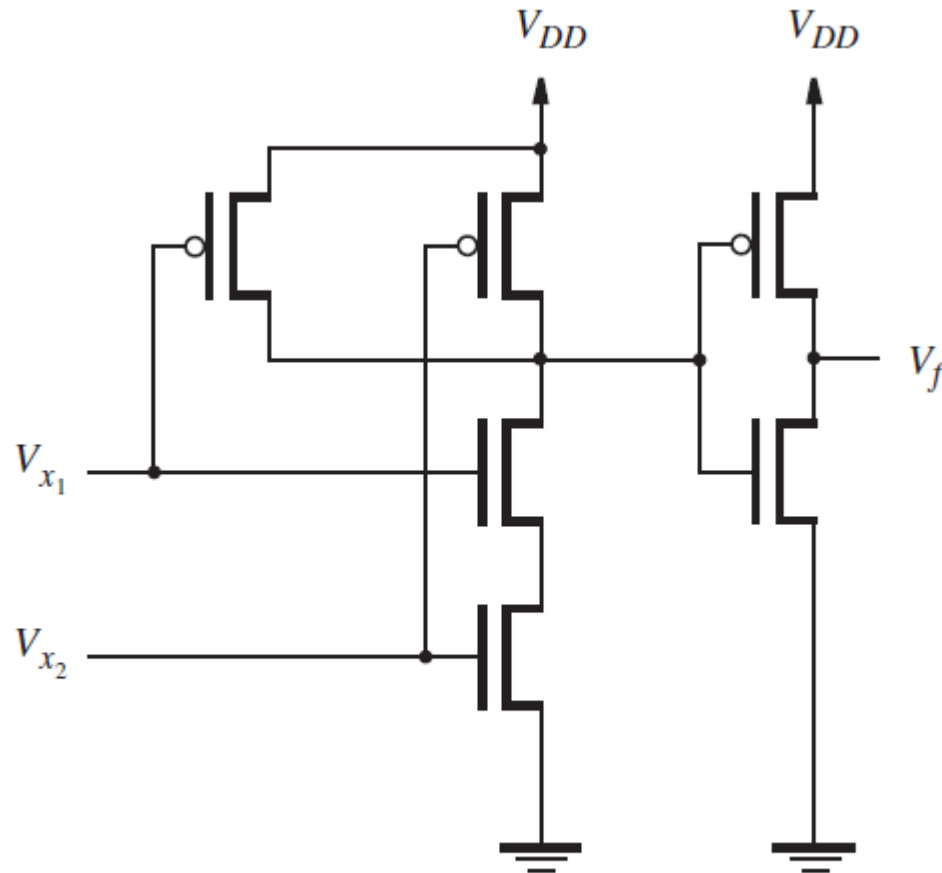
Función



Diagrama de tiempos



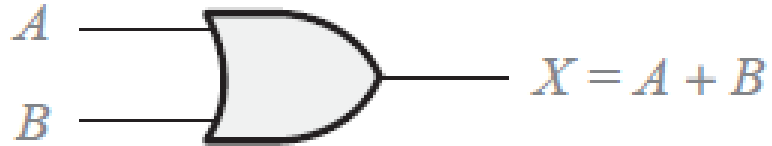
Implementación Básica de Compuertas: AND



Realización CMOS de una compuerta AND.

Compuerta OR (Suma Lógica)

Símbolo y Expresión Funcional



Una **compuerta OR** genera un nivel ALTO a la salida cuando cualquiera de sus entradas está a nivel ALTO.

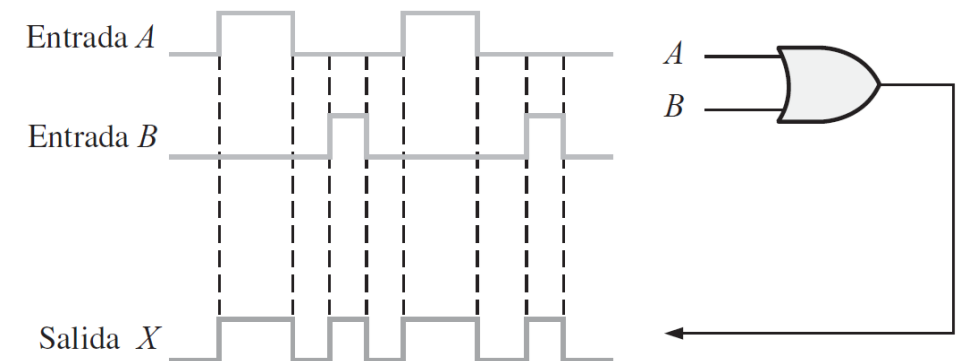
Tabla de Verdad

Entradas		Salida
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1
1 = ALTO, 0 = BAJO		

Función

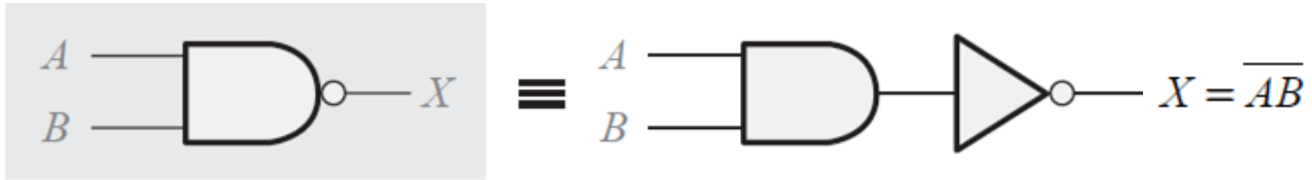


Diagrama de tiempos



Compuerta NAND (Producto Lógico Negado)

Símbolo y Expresión Funcional



La **compuerta NAND** genera una salida a nivel BAJO sólo cuando todas las entradas están a nivel ALTO.

Tabla de Verdad

Entradas		Salida
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0
1 = ALTO, 0 = BAJO		

Función

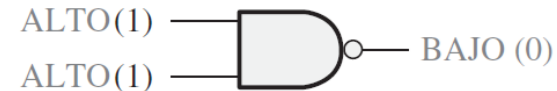
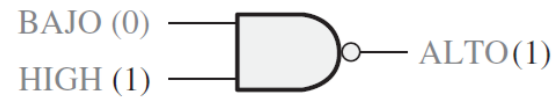
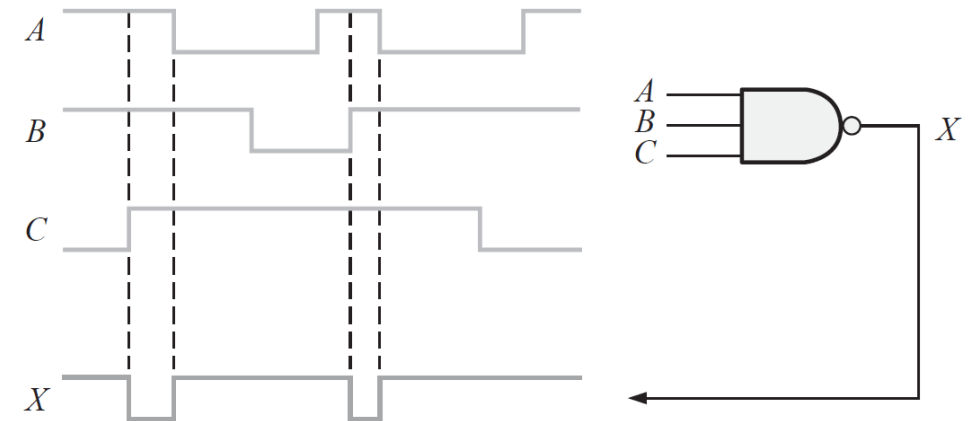
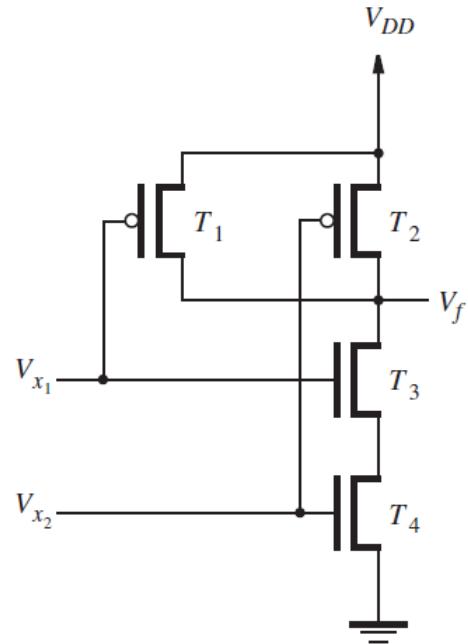


Diagrama de tiempos



Implementación Básica de Compuertas: NAND/NOR

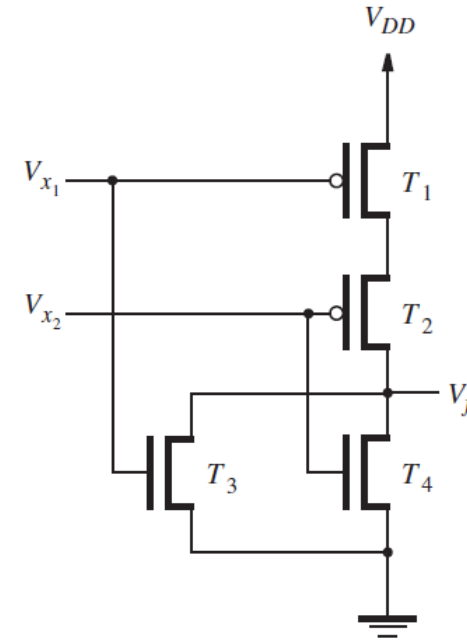


a) Circuito

x_1	x_2	T_1	T_2	T_3	T_4	f
0	0	on	on	off	off	1
0	1	on	off	off	on	1
1	0	off	on	on	off	1
1	1	off	off	on	on	0

b) Tabla de verdad y estados de transistor

Realización CMOS de una compuerta NAND.



a) Circuito

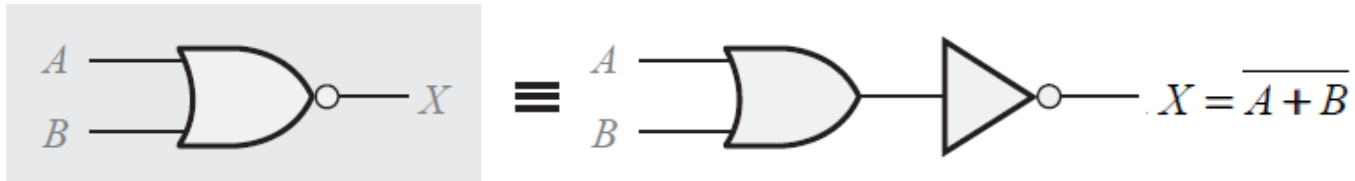
x_1	x_2	T_1	T_2	T_3	T_4	f
0	0	on	on	off	off	1
0	1	on	off	off	on	0
1	0	off	on	on	off	0
1	1	off	off	on	on	0

b) Tabla de verdad y estados de transistor

Realización CMOS de una compuerta NOR.

Compuerta NOR (Suma Lógica Negada)

Símbolo y Expresión Funcional



La **compuerta NOR** genera una salida a nivel BAJO cuando cualquiera de sus entradas está a nivel ALTO

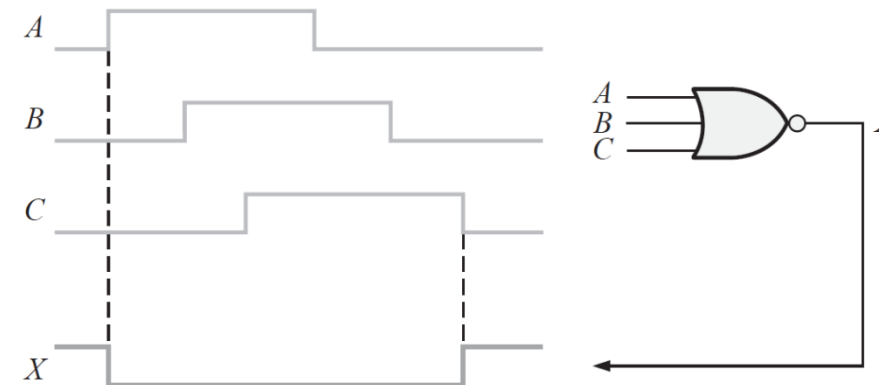
Tabla de Verdad

Entradas		Salida
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0
1 = ALTO, 0 = BAJO		

Función

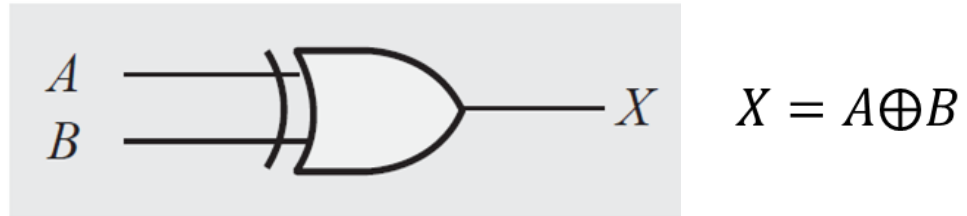


Diagrama de tiempos



Compuerta XOR (Suma Lógica Exclusiva)

Símbolo y Expresión Funcional



La salida de una **compuerta OR-exclusiva** se pone a nivel ALTO sólo cuando las entradas en alto son una cantidad impar.

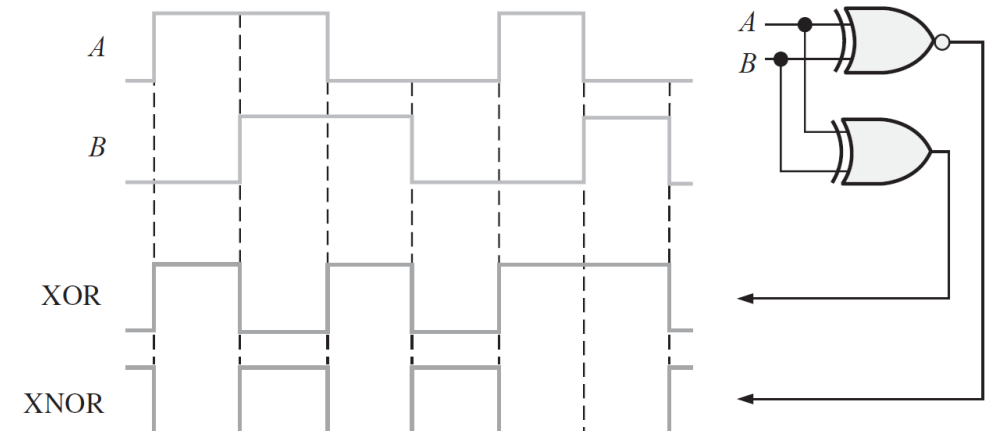
Tabla de Verdad

Entradas		Salida
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Función

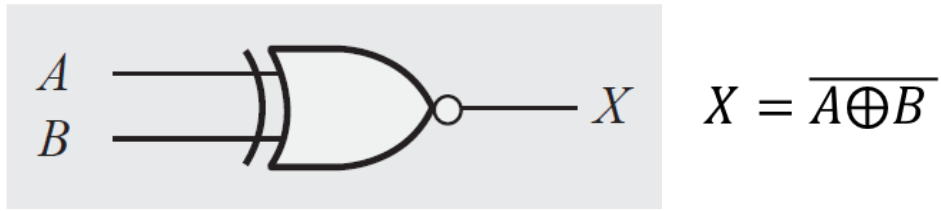


Diagrama de tiempos



Compuerta XNOR (Suma Lógica Exclusiva Negada)

Símbolo y Expresión Funcional



La salida de una **compuerta OR-exclusiva** se pone a nivel ALTO sólo cuando las entradas en alto son una cantidad par.

Función

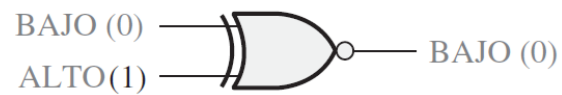
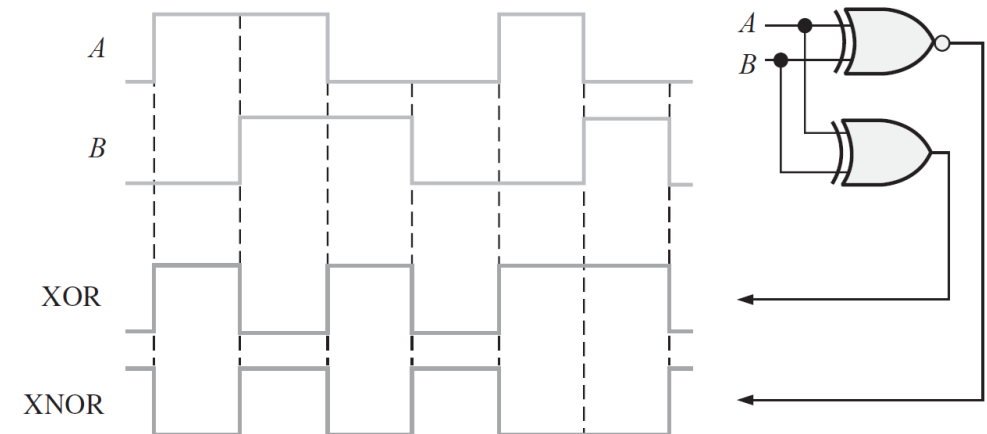


Tabla de Verdad

Entradas		Salida
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

Diagrama de tiempos



Álgebra de Boole

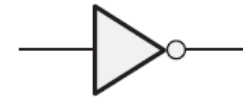
- 1854 - Boole publica la idea de “álgebra lógica”
- 1938 – Claude Shannon la aplica por primera vez al estudio de los circuitos digitales.
- El álgebra de Boole es una forma adecuada y sistemática de expresar y analizar las operaciones de los circuitos lógicos.

Álgebra de Boole – Definición ^(1/2)

- $(\{0,1\}, -, +, .)$ donde se define:

- Operación Complemento: $X = \bar{A}$

Interpretación en circuitos: NOT



- Operación Suma:

$$X = A + B$$

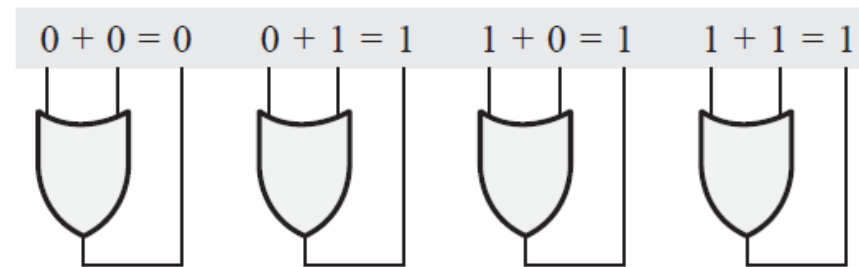
$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 1$$

Interpretación en circuitos: OR

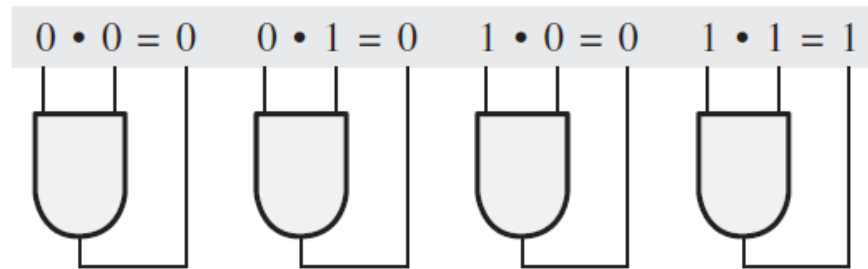


Álgebra de Boole – Definición ^(2/2)

- Operación Producto: $X = AB$

$$\begin{array}{l} 0 \cdot 0 = 0 \\ 0 \cdot 1 = 0 \\ 1 \cdot 0 = 0 \\ 1 \cdot 1 = 1 \end{array}$$

Interpretación en circuitos: AND



Axiomas – Leyes conmutativas

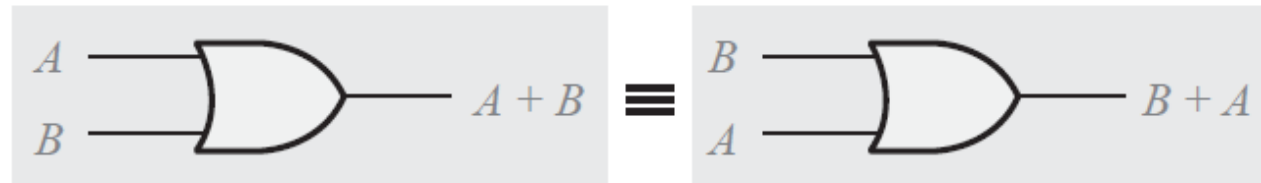
- *Producto:* $AB = BA$

Interpretación en circuitos



- *Suma:* $A + B = B + A$

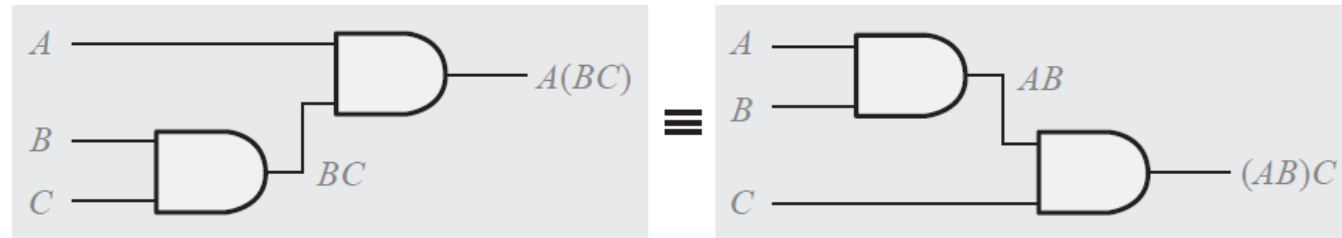
Interpretación en circuitos



Axiomas – Leyes asociativas

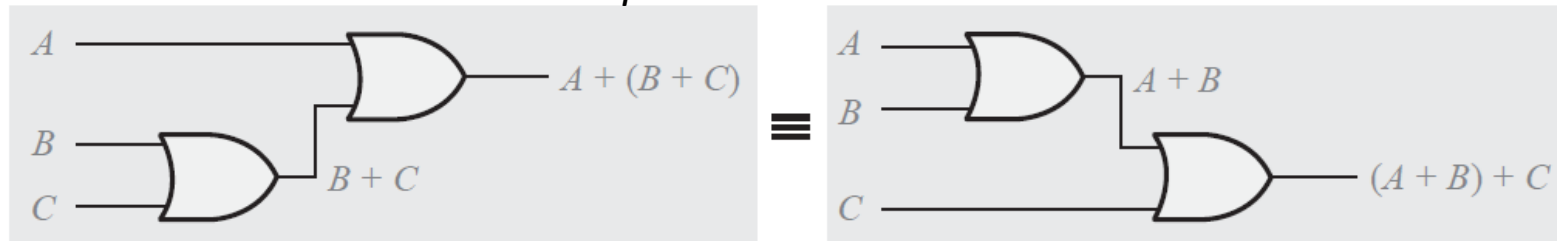
- *Producto:* $A(BC) = (AB)C$

Interpretación en circuitos



- *Suma:* $A + (B + C) = (A + B) + C$

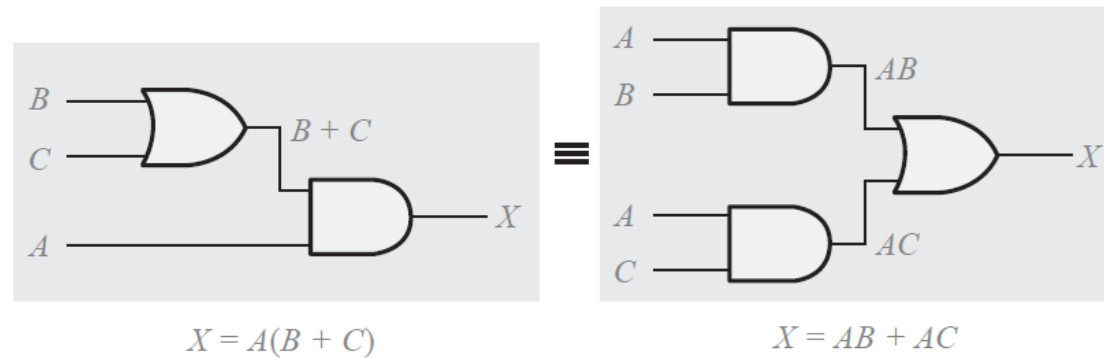
Interpretación en circuitos



Axiomas – Leyes distributivas

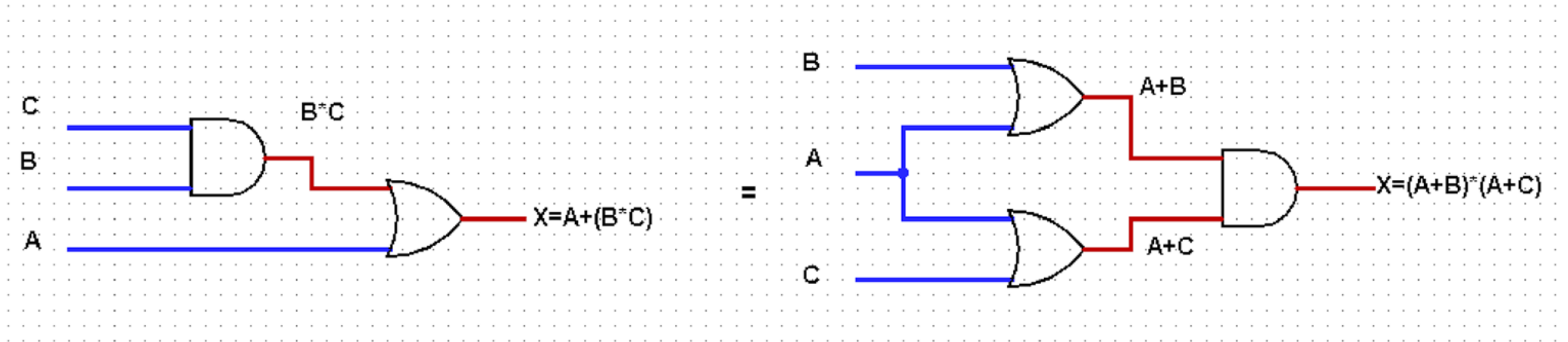
- *Producto respecto a la suma:* $A(B + C) = AB + AC$

Interpretación en circuitos



Axiomas – Leyes distributivas

- *Suma respecto al producto:* $A + (B * C) = (A + B)(A + C)$



Axiomas – Neutros y Complementos

- Neutro del producto:

$$A \cdot 1 = A$$

- Neutro de la suma:

$$A + 0 = A$$

- Complemento para el producto:

$$A \cdot \overline{A} = 0$$

- Complemento para la suma:

$$A + \overline{A} = 1$$

Axiomas – Reglas útiles para manipulación

$$1. A + 0 = A$$

$$2. A + 1 = 1$$

$$3. A \cdot 0 = 0$$

$$4. A \cdot 1 = A$$

$$5. A + A = A$$

$$6. A + \overline{A} = 1$$

$$7. A \cdot A = A$$

$$8. A \cdot \overline{A} = 0$$

$$9. \overline{\overline{A}} = A$$

$$10. A + AB = A$$

$$11. A + \overline{A}B = A + B$$

$$12. (A + B)(A + C) = A + BC$$

A , B o C pueden representar una sola variable o una combinación de variables.

En realidad la 1, 4, 6 y 8 son los axiomas de neutros y complementos.

Se pueden demostrar usando los axiomas (o reglas ya demostradas) o mediante una tabla de verdad.

Ejemplo – Demostración Regla 10: $A + AB = A$

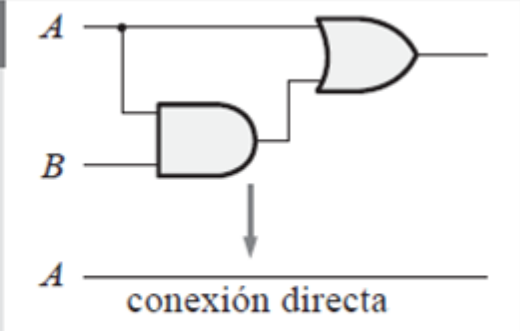
Demostración usando reglas

$$\begin{aligned} A + AB &= A(1 + B) && \text{Sacar factor común (ley distributiva)} \\ &= A \cdot 1 && \text{Regla 2: } (1 + B) = 1 \\ &= A && \text{Regla 4: } A \cdot 1 = A \end{aligned}$$

Demostración con tabla de verdad y circuito simplificado

A	B	AB	$A + AB$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

↑ igual ↑



Ejemplo a Desarrollar Regla 11: $A + \bar{A}B$ (1/4)

Aplicando las reglas del álgebra de Boole operar sobre la expresión para simplificarla:

$$A + \bar{A}B$$

Demostrar el resultado con la tabla de la verdad y mostrar el circuito simplificado

Ejemplo a Desarrollar Regla 11: $A + \bar{A}B$ (2/4)

Demostración usando reglas

$$A + \bar{A}B = (A + AB) + \bar{A}B \quad \text{Regla 10: } A = A + AB$$

$$= (AA + AB) + \bar{A}B \quad \text{Regla 7: } A = AA$$

$$= AA + AB + \bar{A}B + A\bar{A} \quad \text{Regla 8: sumar } A\bar{A} = 0$$

$$= A(A + \bar{A}) + B(A + \bar{A}) \quad \text{Saco factor comun 1y4 y 2y3}$$

$$= A.1 + B.1 \quad \text{Regla 6: } A + \bar{A} = 1$$

$$A + \bar{A}B = A + B$$

Ejemplo a Desarrollar Regla 11: $A + \bar{A}B$ (3/4)

Demostración usando reglas

$$A + \bar{A}B = (A + AB) + \bar{A}B \quad \text{Regla 10: } A = A + AB$$

$$= (AA + AB) + \bar{A}B \quad \text{Regla 7: } A = AA$$

$$= AA + AB + \bar{A}B + A\bar{A} \quad \text{Regla 8: sumar } A\bar{A} = 0$$

$$= A(A + \bar{A}) + B(A + \bar{A}) \quad \text{Saco factor comun 1y4 y 2y3}$$

$$= A.1 + B.1 \quad \text{Regla 6: } A + \bar{A} = 1$$

$$A + \bar{A}B = A + B$$

Demostración con tabla de verdad

A	B	$\bar{A}B$	$A + \bar{A}B$	$A + B$
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1

↑ igual ↑

Ejemplo a Desarrollar Regla 11: $A + \bar{A}B = A + B$ (4/4)

Demostración usando reglas

$$A + \bar{A}B = (A + AB) + \bar{A}B \quad \text{Regla 10: } A = A + AB$$

$$= (AA + AB) + \bar{A}B \quad \text{Regla 7: } A = AA$$

$$= AA + AB + \bar{A}B + A\bar{A} \quad \text{Regla 8: sumar } A\bar{A} = 0$$

$$= A(A + \bar{A}) + B(A + \bar{A}) \quad \text{Saco factor comun 1y4 y 2y3}$$

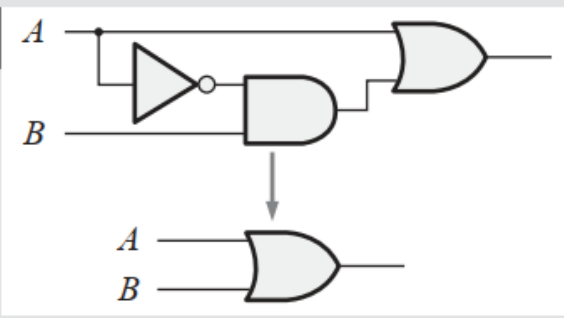
$$= A.1 + B.1 \quad \text{Regla 6: } A + \bar{A} = 1$$

$$A + \bar{A}B = A + B$$

Demostración con tabla de verdad y circuito simplificado

A	B	$\bar{A}B$	$A + \bar{A}B$	$A + B$
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1

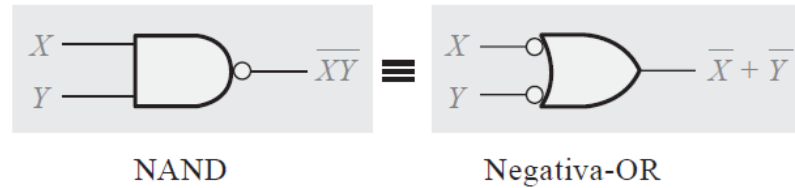
↑ igual ↑



Leyes de De Morgan

- Primer ley: $\overline{XY} = \bar{X} + \bar{Y}$

Interpretación en circuitos

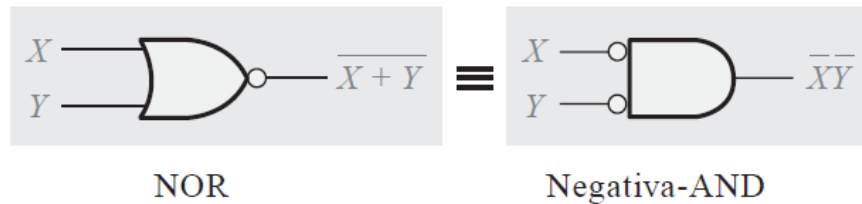


Demostración

Entradas		Salida	
X	Y	$\overline{XY}$	$\bar{X} + \bar{Y}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

- Segunda ley: $\overline{X + Y} = \bar{X}\bar{Y}$

Interpretación en circuitos

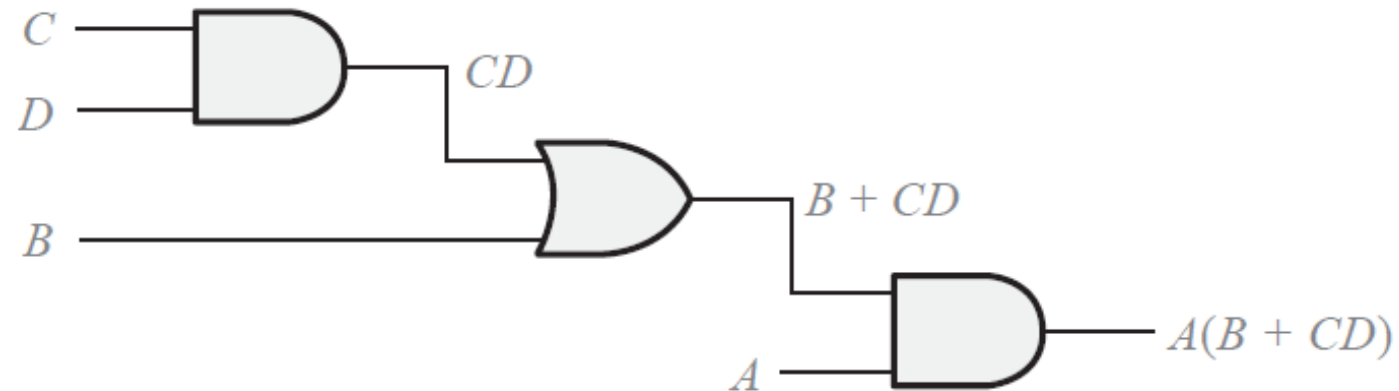


Demostración

Entradas		Salida	
X	Y	$\overline{X + Y}$	$\bar{X}\bar{Y}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

Expresión Booleana de un Circuito Lógico

- Un circuito lógico se puede describir mediante una ecuación booleana.



Método: Ir armando expresiones de izquierda a derecha.

Tabla de Verdad de un Circuito Lógico

- **Con la expresión booleana, podemos armar la tabla de verdad**
 - *Evaluar la expresión para todos los posibles valores de las variables de entrada.*

C	D	B	A	$X=A(B+CD)$
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

Tabla de Verdad a Expresión Booleana

C	D	B	A	X=A(B+CD)
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

- **SOP:** cada término producto es una línea con salida 1.
- **POS:** cada término suma es una línea con salida 0.

$$\text{SOP: } X = \bar{C}\bar{D}\bar{B}A + \bar{C}D\bar{B}A + C\bar{D}\bar{B}A + CD\bar{B}A + CDBA$$

$$\text{POS: } X = (\bar{C} + \bar{D} + \bar{B} + \bar{A})(\bar{C} + \bar{D} + \bar{B} + A)(\bar{C} + \bar{D} + B + \bar{A})(\bar{C} + D + \bar{B} + \bar{A})(\bar{C} + D + \bar{B} + A)(\bar{C} + D + B + \bar{A}) \\ (C + \bar{D} + \bar{B} + \bar{A})(C + \bar{D} + \bar{B} + A)(C + \bar{D} + B + \bar{A})(C + D + \bar{B} + \bar{A})(C + D + B + \bar{A})$$

Simplificación

- **Utilizar las reglas del álgebra booleana para reducir un circuito a una implementación más simple o eficiente.**
 - Con menos compuertas o con compuertas de un tipo dado.
- **Para eso, primero describimos el circuito como una ecuación booleana, y después aplicamos los axiomas y leyes del álgebra booleana para simplificarlo**
 - Por supuesto, esto requiere bastante destreza con las reglas!!!

Simplificación – Ejemplo (1/2)

Simplificar la siguiente expresión utilizando técnicas del álgebra de Boole:

$$AB + A(B + C) + B(B + C)$$

El método que se sigue no es necesariamente el único método posible.

Paso 1. Aplicar la ley distributiva al segundo y tercer término del siguiente modo:

$$AB + AB + AC + BB + BC$$

Paso 2. Aplicar la regla 7 ($BB = B$) al cuarto término.

$$AB + AB + AC + B + BC$$

Paso 3. Aplicar la regla 5 ($AB + AB = AB$) a los dos primeros términos.

$$AB + AC + B + BC$$

Paso 4. Aplicar la regla 10 ($B + BC = B$) a los dos últimos términos.

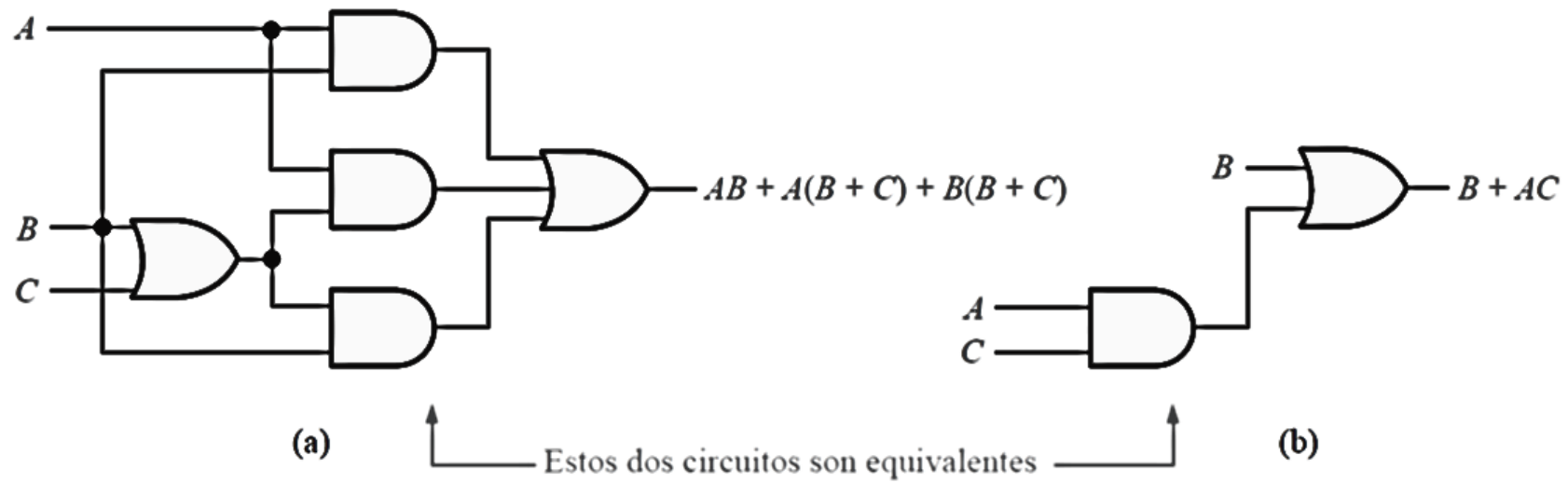
$$AB + AC + B$$

Paso 5. Aplicar la regla 10 ($AB + B = B$) al primero y tercer término.

$$B + AC$$

En este punto, la expresión ya no puede seguir simplificándose. Según vaya adquiriendo experiencia en la aplicación del álgebra de Boole, podrá combinar muchos de los pasos individuales.

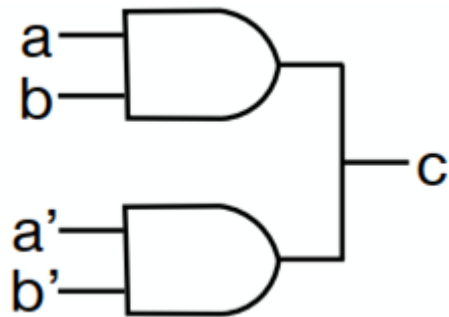
Simplificación – Ejemplo (2/2)



Interpretación en circuitos

Tercer Estado Lógico: Alta Impedancia (hi-Z) ^(1/2)

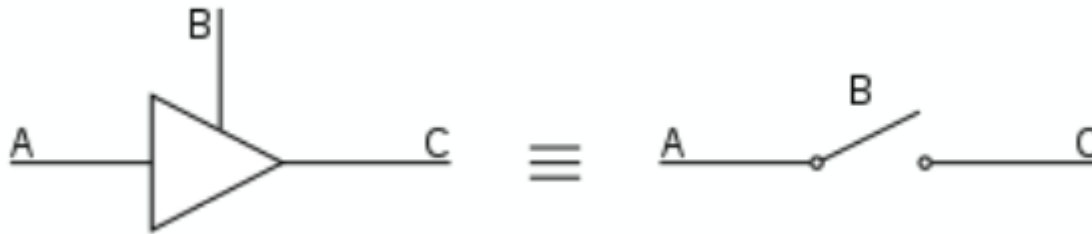
- ¿Cómo se conectan compuertas para hacer circuitos?
 - Las salidas con las entradas: sin problemas.
 - Las entradas siempre deben estar conectadas a un nivel válido.
 - Las salidas no pueden conectarse directamente entre sí. Salvo que se trate de compuertas con el Tercer Estado Lógico (alta impedancia).



a	b	a'	b'	c	
0	0	0	0	0	
0	0	0	1	0	
0	0	1	0	0	
0	0	1	1	☠	corto circuito
0	1	0	0	0	
0	1	0	1	0	
0	1	1	0	0	
0	1	1	1	☠	corto circuito

Tercer Estado Lógico: Alta Impedancia (hi-Z) ^(2/2)

- El tercer estado lógico distingue un estado de un circuito en el que no es posible observar ninguno de los estados lógicos 0 ó 1.
 - Este estado se denomina Alta Impedancia (Hi-Z)
 - Se puede interpretar como una desconexión eléctrica de la compuerta.
- La utilización de buffers de tres estados permite que varios dispositivos compartan una misma salida si se tiene la precaución de que solo uno de estos tenga habilitada la salida.



A	B	C
X	0	hi-Z
0	1	0
1		1

Ejercicio de Desarrollo (1/4)

- **Requerimiento:**

- Diseñe un circuito que compruebe si un número expresado en complemento a 2 (de 3 bits de precisión) es negativo o par. Se deben plantear dos salidas independientes para cada una de las corroboraciones.

- **Pasos:**

- Armar tabla de la verdad
- Simplificar usando las reglas del álgebra de Boole
- Dibujar el circuito de compuertas simplificado

Ejercicio de Desarrollo (2/4)

$$nro = b_2 b_1 b_0$$

0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	0	0
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	1	0

Utilizo SOP= sumas de productos, primero para salida neg

$$neg = b_2 \overline{b_1} \overline{b_0} + b_2 \overline{b_1} b_0 + b_2 b_1 \overline{b_0} + b_2 b_1 b_0$$

$$neg = b_2 \overline{b_1} \cdot (\overline{b_0} + b_0) + b_2 b_1 \cdot (\overline{b_0} + b_0)$$

$$neg = b_2 \overline{b_1} \cdot (1) + b_2 b_1 \cdot (1)$$

$$neg = b_2 \overline{b_1} + b_2 b_1$$

$$neg = b_2 \cdot (\overline{b_1} + b_1)$$

$$neg = b_2 \cdot (1)$$

$$neg = b_2$$

Ejercicio de Desarrollo (3/4)

$$nro = b_2 b_1 b_0$$

0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	0	0
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	1	0

Utilizo SOP= sumas de productos, ahora para salida par

$$par = \overline{b_2} \overline{b_1} \overline{b_0} + \overline{b_2} b_1 \overline{b_0} + b_2 \overline{b_1} \overline{b_0} + b_2 b_1 \overline{b_0}$$

$$par = \overline{b_1} \overline{b_0} \cdot (\overline{b_2} + b_2) + b_1 \overline{b_0} \cdot (\overline{b_2} + b_2)$$

$$par = \overline{b_1} \overline{b_0} \cdot (1) + b_1 \overline{b_0} \cdot (1)$$

$$par = \overline{b_1} \overline{b_0} + b_1 \overline{b_0}$$

$$par = \overline{b_0} \cdot (\overline{b_1} + b_1)$$

$$par = \overline{b_0} \cdot (1)$$

$$par = \overline{b_0}$$

Ejercicio de Desarrollo (4/4)

$$nro = b_2b_1b_0$$

0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	0	0
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	1	0

$$neg = b_2$$

$$par = \overline{b_0}$$

$$b_2 \text{ ————— } neg$$

$$b_0 \text{ — } \triangle \text{ — } par$$



Fin: Funciones Lógicas y Álgebra de Boole